

Volume 48

Sentinel Developer Ecosystem & Plugin SDK

Status: Strategic Platform Layer

Vision

Project Sentinel shall become a platform that other developers, consultants, enterprises, universities, and security researchers can extend safely.

The core platform remains stable.

Innovation happens through plugins, SDKs, rule packs, mission templates, integrations, and dashboards.

Why this matters

No single team can build every security capability.

Project Sentinel needs an ecosystem.

Examples:

- A consultant builds a custom compliance pack.
- A university builds a teaching module.
- A bank builds internal policy rules.
- A government agency builds offline threat intelligence imports.
- A bug bounty researcher builds an authorized recon workflow.
- A company builds a connector for its internal CMDB.

All without modifying the kernel.

Extension Types

Project Sentinel supports multiple extension types.

Plugins

Each extension is signed, versioned, permission-scoped, and auditable.

Plugin Philosophy

Plugins are not random scripts.

Plugins are first-class platform citizens.

Every plugin has:

- Plugin ID
 - Name
 - Version
 - Author
 - Capabilities
 - Required permissions
 - Supported platform version
 - Dependencies
 - Signature
 - Trust level
 - Documentation
 - Test suite
-

Plugin Object

Every plugin becomes a Sentinel Object.

Plugin ID

SDK Layers

Project Sentinel will provide several SDK layers.

Python SDK

For building engines, collectors, connectors, and automation.

REST SDK

For external systems.

GraphQL SDK

For advanced queries and integrations.

Rule SDK

For writing testable rules.

Mission SDK

For creating mission templates.

Workflow SDK

For automation.

Object SDK

For creating new object types.

AI Provider SDK

For integrating local or cloud AI models.

Python SDK Example

Future developers should be able to write plugins like this:

```
from sentinel.sdk import SentinelPlugin, Evidence
```

Simple.

Readable.

Maintainable.

Plugin Permissions

Plugins must request explicit permissions.

Examples:

NetworkRead

No plugin receives unlimited access by default.

Plugin Sandboxing

Plugins should execute in restricted environments where possible.

Controls may include:

- Permission scopes

- Resource limits
- File system boundaries
- Network restrictions
- Runtime isolation
- Signature verification

Security extensions must not weaken platform security.

Plugin Lifecycle

Submitted

Every stage is auditable.

Marketplace Architecture

A future Sentinel Marketplace can distribute:

- Official extensions
- Community extensions
- Enterprise-private extensions
- Government-private extensions

Marketplace items are not blindly trusted.

They must pass validation.

Extension Validation

Every extension must include:

- Manifest
- Permissions
- Documentation
- Tests
- Compatibility metadata
- Security notes
- Version history

The platform validates before installation.

Rule Packs

Rule packs contain versioned rules.

Examples:

- Web Security Baseline
 - Endpoint Security Baseline
 - Mobile Privacy Baseline
 - Banking Security Baseline
 - Government Hardening Baseline
 - ISO 27001 Evidence Pack
-

Compliance Packs

Compliance packs map evidence to controls.

Examples:

- ISO 27001
- NIST CSF
- CIS Controls
- PCI DSS
- SOC 2
- Internal company frameworks

Mappings remain transparent and versioned.

Mission Templates

Developers can create reusable missions.

Examples:

Verify Finance Laptops

Templates reduce complexity for users.

Dashboard Packs

Organizations can install dashboards tailored for:

- CISO
 - SOC
 - Endpoint team
 - Mobile team
 - Cloud team
 - Compliance team
 - Home users
-

Report Templates

Reports can be customized by audience:

- Executive report
 - Technical report
 - Compliance report
 - Evidence report
 - Incident report
 - Bug bounty report
 - Home security report
-

Language Packs

Because Project Sentinel must support English, Norwegian, and additional languages.

Language packs include:

- UI strings
- Report phrases
- Security terminology
- Help text
- Recommendation explanations

No hardcoded user-facing text.

AI Provider Extensions

AI providers can be swapped.

Examples:

- Local model
- Private enterprise model
- Cloud provider
- Future AI engine

All AI providers must follow the Evidence-First architecture.

Connector Extensions

Connectors integrate with:

- Identity providers
- Cloud providers
- Ticketing systems
- CMDB
- SIEM
- EDR
- MDM
- Email gateways
- Notification systems

Project Sentinel should coexist with existing enterprise tools.

NEW CORE ENGINE

Extension Trust Engine

Evaluates extension trust based on:

- Signature
- Author
- Permissions
- Code review status
- Runtime behavior
- Version history
- Community reputation
- Enterprise approval

Trust is transparent and visible.

NEW CORE ENGINE

Plugin Compatibility Engine

Before upgrade, the platform checks:

- API compatibility
- Object schema compatibility
- Permission changes
- Deprecated methods
- Required migrations

This prevents breaking enterprise deployments.

NEW CORE ENGINE

Extension Observatory

Administrators can see:

- Active plugins
- Resource usage
- Permissions
- Errors
- Events emitted
- Evidence created
- API calls
- Version status

No plugin operates invisibly.

PS-ADR-0043

Extensible by Design

Status: Accepted

Decision

Project Sentinel shall expose a secure plugin and SDK architecture that allows new capabilities to be added without modifying the core kernel.

Rationale

A cybersecurity operating environment must evolve faster than any single engineering team can build. A controlled extension ecosystem enables innovation while preserving platform stability, security, and maintainability.

New Long-Term Vision

Sentinel Ecosystem

Project Sentinel should eventually become more than a product.

It becomes an ecosystem:

Core Platform

This creates long-term growth without compromising the core architecture.

Important principle

We must never allow plugins to create legacy problems.

Therefore:

The kernel remains stable. Extensions adapt to the kernel, not the other way around.

That protects the architecture for decades.

Looking Ahead

The next volume should be **Volume 49 - Sentinel Python Runtime Architecture**.

This is where we move closer to implementation.

We will define:

- Python package structure

- Core service boundaries
- Async architecture
- Worker runtime
- Event handling
- Plugin loading
- Testing structure
- Configuration management
- Dependency management
- Logging
- Error handling
- Security hardening

This will be one of the most important practical volumes because it turns the blueprint from architecture into actual implementation strategy.

=====[USER]=====

good and continue

=====[ASSISTANT]=====

Excellent. We are now entering what I consider the beginning of **Phase 3**.

Everything we've designed so far answers:

What should Project Sentinel do?

From now on we answer:

How do we engineer it so it can still be running 20 years from now?

As we agreed from the beginning:

- Pure Python
- No legacy code
- Modular
- Replaceable engines
- Clean architecture
- Easy to understand
- Easy to maintain
- Easy to extend

This volume is probably the most important one for implementation.

PROJECT SENTINEL BLUEPRINT

Volume 49